

Skripta: IoT a vstavané systémy

Kolektív

May 19, 2026

Contents

1	Úvod	3
2	Vstavané systémy — podrobný prehľad	5
2.1	Architektúra systému	5
2.2	Výber mikrokontroléra	5
2.3	Komunikačné protokoly a periférie	5
2.4	Tok dát v zariadení	7
2.5	WS2812B a riadiaci protokol	7
2.6	Napájanie a správa energie	7
2.7	Časovanie a real-time požiadavky	8
2.8	Softvérové modely	8
2.9	Testovanie a ladenie	8
2.10	Bezpečnosť a spoľahlivosť	8
2.11	Nasadenie a údržba	8
2.12	Zhrnutie	8
3	IoT, siete a topológie	9
3.1	IoT architektúra — vrstvy a komponenty	9
3.2	Sieťové topológie	9
3.3	Praktické scenáre a odôvodnenie	10
3.4	Komunikačné protokoly	11
3.5	Edge vs. cloud processing	11
3.6	Brána (gateway) — funkcie	11
3.7	Škálovanie a správa	11
3.8	Bezpečnostné požiadavky	11
3.9	Príklady nasadení	11
3.10	Zhrnutie	12
4	ESP32 – prehľad platformy	13
5	MicroPython pre vstavané systémy	15
6	Príklady podľa zadania	17
7	Servisný test dosky	17
8	Interakcia s tlačidlami: hra hádania	20
9	Digitálne riadenie RGB LED	21
10	PWM riadenie jasu RGB LED	22

11 Prerušenia a meranie dĺžky stlačenia	23
12 Sériové LED WS2812B	24
13 PWM bzučiak a krátka melódia	25
14 Analógové meranie svetla	26
15 Senzor DHT: teplota a vlhkosť	27
16 Wi-Fi prístupový bod a webové ovládanie	28
17 Odosielanie dát do ThingSpeak	30
18 MQTT publish/subscribe komunikácia	31

1 Úvod

Tento text predstavuje základné pojmy a kontext pre prácu s IoT a vstavanými systémami. IoT (Internet vecí) označuje súbor zariadení, ktoré sú vybavené senzormi, výpočtovým jadrom a sieťovou konektivitou s cieľom zbierať, spracovávať a zdieľať informácie. Vstavané systémy sú pritom výpočtové jednotky navrhnuté tak, aby spoľahlivo a efektívne vykonávali úlohy so špecifickými obmedzeniami — nízka spotreba, malá veľkosť, limitovaná pamäť a často pevne stanovené rozhrania.

Na úrovni návrhu sa zariadenie typicky skladá z troch vrstiev: (1) senzorickej vrstvy (zber údajov), (2) riadiacej vrstvy (spracovanie a rozhodovanie) a (3) komunikačnej/výstupnej vrstvy (odovzdanie výsledkov alebo spúšťanie akcií). Táto modularita uľahčuje navrhnuť systém, ktorý je rozšíriteľný, testovateľný a bezpečný.

V tejto knihe najprv uvedieme základné princípy vstavaných systémov a hardvérové komponenty, potom prejdeme k praktickej platforme ESP32 a programovaniu v MicroPythone. Následné kapitoly obsahujú návod na nasadenie, sieťové topológie pre IoT, návrh energeticky efektívnych riešení a súbor praktických príkladov a testovacích postupov.

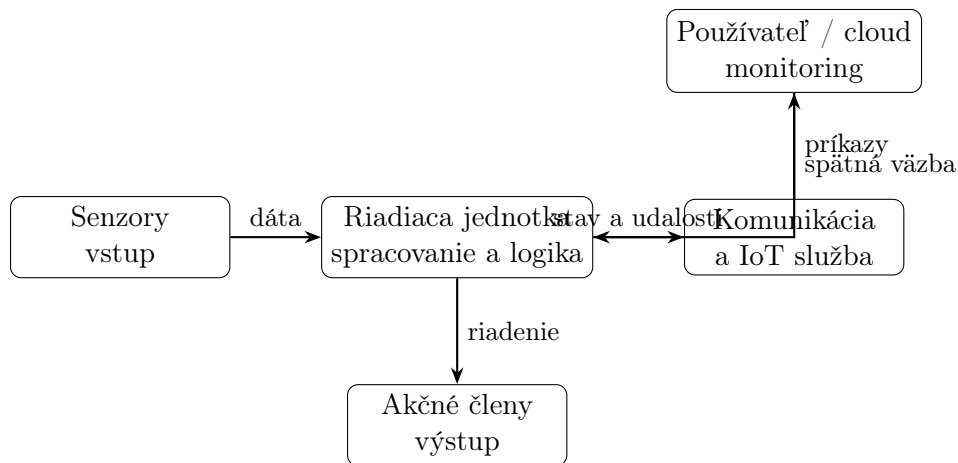


Figure 1: Zjednodušená schéma vstavaného IoT riadiaceho systému.

Kľúčové koncepty vstavaných systémov sa dajú čítať aj z pohľadu hardvéru a z pohľadu softvéru.

Hardvérové prvky

Mikrokontrolér je iba jedna časť celku. V praxi ho dopĺňajú senzory, akčné členy, pamäť, napájanie, ochranné prvky a komunikačné rozhrania. Medzi najčastejšie periférie patria ADC pre analógové merania, DAC pre analógový výstup, PWM pre riadenie jasů alebo výkonu, časovače, čítače impulzov, GPIO a jednotky pre sériovú komunikáciu.

Komunikačné protokoly a periférie

Na doske plošných spojov sa často používajú nasledujúce rozhrania:

- **I2C** pre senzory, expandéry vstupov a displeje; používa dve vodiče a adresovanie zariadení.
- **SPI** pre rýchlejšiu komunikáciu s displejmi, pamäťami alebo prevodníkmi.
- **I2S** pre digitálny zvuk, mikrofóny a audio prevodníky.
- **UART** pre servisnú komunikáciu, ladenie a prepojenie s inými modulmi.

- **PWM** pre stmievanie LED, riadenie motorov a generovanie tónov.
- **ADC a DAC** pre prácu s analógovým signálom.
- **TWAI/CAN, SDIO, USB** alebo **RMT** tam, kde je potrebný robustnejší prenos alebo presné časovanie.

Tieto rozhrania sú dôležité preto, že prepojujú mikrokontrolér s reálnym svetom. Výber konkrétneho protokolu závisí od rýchlosti, počtu zariadení na zbernici, požiadaviek na časovanie a od typu pripojenej periférie.

Softvérový model

Vstavaný firmvér môže bežať bez operačného systému, kde hlavný program obieha v slučke a priebežne obsluhuje vstupy, alebo nad RTOS, ktorý rozdeľuje úlohy na samostatné vlákna a plánuje ich beh. Bez ohľadu na zvolený model je dôležité správne spracovanie prerušení, časovačov, oneskorení a správy energie.

Prečo je to dôležité pre IoT

IoT zariadenie musí často kombinovať lokálnu logiku s komunikáciou do siete. To znamená, že okrem samotného merania a riadenia musí vedieť aj ukladať stav, posilať dáta na server, prijímať príkazy a bezpečne pracovať s identitou zariadenia. V ďalších kapitolách sa preto prepája hardvér, sieťová komunikácia a programovanie v MicroPythone na platforme ESP32.

Nasledujúce kapitoly

Po úvode nasleduje podrobná kapitola venovaná návrhu vstavaných systémov (architektúra, výber mikrokontroléra, napájanie, časovanie a softvérové modely). Ďalšia časť sa zameriava na IoT a siete — popíšeme bežné topológie, komponenty brány (gateway), edge vs. cloud spracovanie a bezpečnostné aspekty. Praktická časť potom ukáže prácu s platformou ESP32 a MicroPythonom, vrátane inštalácie cez Thonny a ukážkových projektov.

Napájanie a spoľahlivosť

Pri reálnom nasadení treba riešiť aj spotrebu, reset správanie, watchdog, ochranu proti výpadkom napájania a vhodné spánkové režimy. V batériových zariadeniach býva dôležitý aj návrh zobúdzacích podmienok a minimalizácia spotreby v nečinnosti.

Bezpečnosť

S rastúcim počtom pripojených zariadení rastie aj význam bezpečnosti. Do návrhu patrí autentifikácia zariadení, ochrana komunikácie, rozumné spravovanie kľúčov a kontrola toho, kto môže meniť stav zariadenia alebo čítať jeho dáta.

V literatúre nájdete prehľad architektúr a konkrétnych návrhov od výrobcov mikrokontrolérov Wolf 2012.

V literatúre nájdete prehľad architektúr a konkrétnych návrhov od výrobcov mikrokontrolérov Wolf 2012.

2 Vstavané systémy — podrobný prehľad

Táto kapitola rozpracováva kľúčové aspekty návrhu vstavaných systémov. Je určená pre inžinierov a študentov, ktorí potrebujú praktické rady pri výbere komponentov, návrhu napájania, riadení časovania a správe softvéru.

2.1 Architektúra systému

Základom návrhu je pochopenie funkčných blokov: senzory, spracovanie, pamäť, komunikácia a akčné členy. Pri menších systémoch postačuje jednoduchá jednovrstvová architektúra, pri zložitejších sa uplatní viacvrstvový prístup s jasným oddelením zberu dát, lokálneho spracovania a odosielania do cloudu.

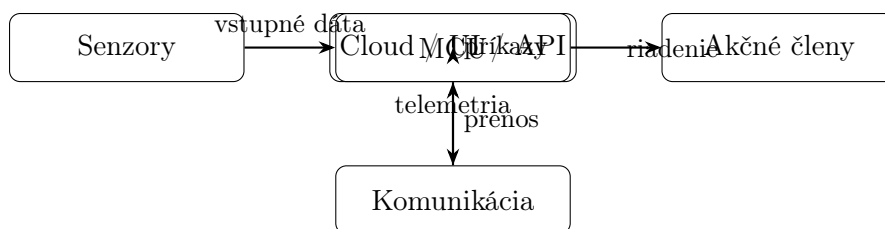


Figure 2: Všeobecná bloková schéma vstavaného IoT zariadenia.

Pri návrhu treba zväžiť:

- požadovaný výpočtový výkon a spotrebu,
- dostupnú pamäť (RAM a flash),
- rozhrania pre periférie (I2C, SPI, UART, ADC, PWM),
- potrebu reálneho času alebo plánovania úloh (RTOS vs. jednoduchá slučka),
- požiadavky na spoľahlivosť a dostupnosť.

2.2 Výber mikrokontroléra

Pri výbere MCU zväžte prenosové rozhrania, dostupné periférie, podporu voľby napájania, CAD a vývojové nástroje. Dôležité sú tiež faktory ako dostupnosť knižníc, komunita a kompatibilita so zamýšľaným jazykom (MicroPython, C/C++).

Pri projektoch IoT bývajú obľúbené MCU s integrovanou Wi-Fi/BLE (napr. ESP32), pre priemyselné aplikácie zase robustnejšie riešenia s CAN alebo ethernetom.

2.3 Komunikačné protokoly a periférie

Pre vstavané systémy je dôležité rozumieť tomu, čo jednotlivé protokoly prinášajú z pohľadu rýchlosti, zapojenia, adresovania a spoľahlivosti. Výber rozhrania ovplyvňuje nielen hardvér, ale aj zložitosť firmvéru, spotrebu a možnosti rozšírenia.

I2C

I2C je dvojvodičová zbernica založená na linkách SDA a SCL. Typicky sa používa na pripojenie senzorov, malých displejov, RTC modulov a portových expandérov. Jeho výhodou je jednoduché zapojenie a viacero zariadení na jednej zbernici; obmedzením je kratší dosah, nutnosť pull-up rezistorov a citlivosť na kvalitu vodičov pri vyšších rýchlostiach.

V praxi je vhodný tam, kde ide o malý počet periférií na jednej doske alebo v blízkom okolí mikrokontroléra. Ak potrebujeme spoľahlivé čítanie viacerých senzorov, I2C je často najjednoduchšia voľba.

SPI

SPI je rýchle sériové rozhranie, ktoré používa signály SCLK, MOSI, MISO a jeden alebo viac výberových pinov CS. Hodí sa pre displeje, pamäte, rýchle prevodníky a periférie, ktoré vyžadujú vyššiu priepustnosť než I2C. Oproti I2C je zapojenie náročnejšie na počet vodičov, ale prenos býva stabilnejší a rýchlejší.

Výhodou SPI je nízka režijná záťaž a dobré časovanie pri čítaní aj zápise. Preto sa používa pri častých aktualizáciách obrazovky, záznamových médiách alebo pri rýchlych dátových prevodníkoch.

UART

UART je asynchrónne sériové rozhranie, ktoré pracuje s dvojicou Tx/Rx a spoločnou zemou. Používa sa na ladenie, sériové konzoly, komunikáciu s modulmi GPS, Bluetooth, GSM alebo inými mikrokontrolérmi. Jeho výhodou je jednoduchosť; nevýhodou je spravidla bod-bod komunikácia bez natívneho adresovania.

UART je vhodný, keď potrebujeme ľahký servisný kanál alebo výmenu príkazov medzi zariadeniami. Pri dlhších vedeniach a rušnom prostredí však treba riešiť úroveň signálu a elektromagnetickú odolnosť.

I2S

I2S sa používa na digitálny audio prenos medzi mikrokontrolérom, mikrofónom, DAC alebo zosilňovačom. Prenos je synchronizovaný samostatným hodinovým signálom, čo je dôležité pri spracovaní zvuku v reálnom čase. V porovnaní s UART alebo I2C je orientovaný na presné časovanie audio dát.

PWM, ADC a DAC

PWM je výstupná technika na simuláciu analógovej úrovne alebo riadenie výkonu. Používa sa na LED, motorčeky a bzučiaky. ADC naopak číta analógové napätie zo senzorov, ako sú fotorezistory, potenciometre alebo tlakové snímače. DAC generuje skutočné analógové napätie, ak ho MCU podporuje.

Tieto bloky sú často súčasťou jedného projektu: ADC číta hodnotu zo senzora, MCU ju spracuje a PWM alebo DAC podľa nej ovláda výstup.

TWAI/CAN, USB, SDIO a RMT

TWAI/CAN sa používa tam, kde je dôležitá robustnosť, viac uzlov a odolnosť voči rušeniu — typicky v automobiloch a priemysle. USB slúži na pripojenie k hostiteľovi, debug alebo emuláciu zariadenia. SDIO je vhodné pri práci s pamäťovými kartami alebo niektorými bezdrôtovými modulmi. RMT využívame tam, kde je potrebné presné generovanie alebo meranie impulzov, napríklad pri LED reťazcoch typu WS2812.

Ako si vybrať správne rozhranie

I2C je vhodné pre jednoduché senzory a malé dosky, SPI pre rýchle periférie s vyššou priepustnosťou, UART pre servis a jednoduché moduly, I2S pre audio a CAN pre odolné priemyselné nasadenia. Výber rozhrania by mal vždy vychádzať z požadovanej rýchlosti, dĺžky vedenia, rušenia, počtu zariadení a dostupných pinov.

2.4 Tok dát v zariadení

Bežný tok dát sa dá opísať ako kruh: senzor zachytí fyzikálnu veličinu, MCU ju spracuje, rozhodne o akcii, pošle výsledok do periférie alebo siete a následne čaká na ďalšiu zmenu stavu. V menších systémoch prebieha tento cyklus v jednej slučke, vo väčších projektoch sa rozdeľuje na úlohy, fronty alebo udalosti.

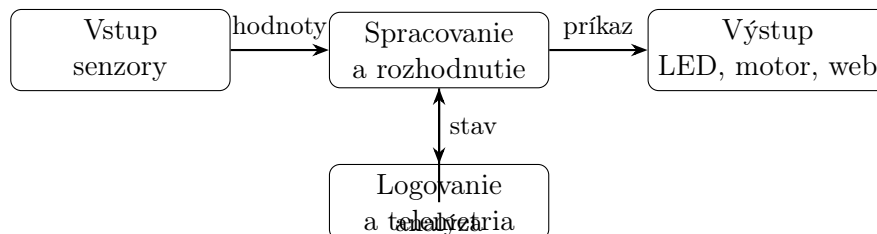


Figure 3: Tok dát v jednoduchom vstavanom systéme.

2.5 WS2812B a riadiaci protokol

WS2812B je adresovateľná RGB LED, ktorá nepoužíva klasické sériové rozhranie typu I2C alebo SPI. Namiesto toho prijíma jednovodičový tok presne časovaných impulzov, kde sa informácia prenáša dĺžkou logickej nuly a jednotky. Každý pixel očakáva 24 bitov dát v poradí G-R-B a po prijatí vlastných bitov posúva zvyšok reťazca ďalej.

Z pohľadu mikrokontroléra je kľúčové presné časovanie: typicky sa vysielajú symbol pre 0 a symbol pre 1 s rozdielnou šírkou pulzu, pričom tolerancia je malá. Preto sa pri implementácii často používa hardvérový periférny blok, napríklad RMT na ESP32, alebo priamo knižnica, ktorá zabezpečí presné generovanie časovania bez prerušovania inými úlohami.

Praktický význam protokolu je v tom, že každý pixel sa správa ako „posuvný register“: prvých 24 bitov si ponechá pre seba a zvyšok odosiela ďalej. Po odoslaní dát je potrebná reset/pause medzera, aby všetky LED zároveň prijali novú farbu. Tento prístup je vhodný pre svetelné pásy, ambientné osvetlenie a stavové indikátory, ale vyžaduje stabilné napájanie a dobré vedenie dátového signálu.

Prečo sa používa RMT alebo špeciálna periféria

Ak by sa časovanie generovalo iba softvérovo, ľahko vzniknú chyby pri prerušení alebo pri vysokej záťaži CPU. Hardvérová periféria udrží presnosť impulzov aj počas iných úloh, čo je dôležité najmä pri dlhších reťazcoch a vyššom počte pixelov.

2.6 Napájanie a správa energie

Napájací návrh často rozhoduje o úspechu projektu. Kľúčové prvky:

- stabilné napájanie s ochranou proti prepätiu a prepólovaniu,
- vhodné DC–DC meniče alebo LDO podľa požiadaviek na účinnosť a šum,
- rezervy pre spúšťacie prúdy (Wi-Fi pri pripojení môže krátkodobo zvýšiť odber),
- návrh batériového režimu: meranie stavu batérie, režimy spánku, zobudenie z prerušení.

Optimalizácia spotreby zahŕňa zníženie aktivít CPU, zhadzovanie periférií do nízkeho spotrebného režimu a minimalizovanie rádiovkej komunikácie (agregácia dát, dlhšie intervaly odosielania).

2.7 Časovanie a real-time požiadavky

Ak je systém závislý na presnom časovaní (riadenie motorov, audio, merania s presným vzorkovaním), je potrebné použiť hardvérové časovače, DMA a prípadne RTOS. IRQ handlers by mali byť stručné a nastavovať príznaky, nie spúšťať dlhé operácie.

2.8 Softvérové modely

Existujú dva základné prístupy: jednoduchá hlavná slučka (superloop) a systém založený na RTOS. Superloop je ľahký a vhodný pre malé úlohy, RTOS poskytuje plánovanie, synchronizáciu a izoláciu úloh, čo pomáha riešiť zložité systémy s viacerými časovo kritickými komponentmi.

Pri MicroPythone je bežné kombinovať jednoduchú štruktúru s časovačmi a asynchrónnym I/O (uasyncio) pre spracovanie viacerých udalostí.

2.9 Testovanie a ladenie

Dôležité sú nasledujúce nástroje a postupy:

- sériová konzola a REPL pre interaktívne skúšanie,
- logging s rôznymi úrovňami (info, warning, error),
- simulácia periférií pri vývoji softvéru,
- jednotkové testy niektorých logických blokov, ak sú možné,
- hardvérové testy: meranie napájacích profilov, testy EMC a teplotné skúšky v extrémoch.

2.10 Bezpečnosť a spoľahlivosť

Bezpečnosť musí byť súčasťou návrhu od začiatku: autentifikácia zariadenia, bezpečné ukladanie kľúčov, šifrovanie komunikácie a pravidelné aktualizácie firmvéru. Spoľahlivosť zahŕňa watchdog, spracovanie chýb, redundantné riešenia tam, kde je to potrebné, a robustné správanie pri výpadku siete.

2.11 Nasadenie a údržba

Pri nasadení hromadných zariadení zvažujeme OTA aktualizácie, monitorovanie stavu zariadení a mechanizmy rollbacku pri zlyhaní aktualizácie. Systém by mal zaznamenávať telemetriu, ktorá pomôže pri diagnostike a plánovaní údržby.

2.12 Zhrnutie

Tento rozšírený prehľad pokrýva najdôležitejšie rozhodnutia, ktoré treba spraviť pri návrhu vstavaného IoT zariadenia. V praktických kapitolách sa zameriame na konkrétne návrhy pre ESP32 a postupy v MicroPythone.

3 IoT, siete a topológie

Táto kapitola sa venuje štruktúre IoT systémov, bežným sieťovým topológiám a spôsobom integrácie lokálnych zariadení s bránami (gateway) a cloudovými službami.

3.1 IoT architektúra — vrstvy a komponenty

Typická IoT architektúra zahŕňa nasledujúce vrstvy:

- Periférna vrstva: senzory a akčné členy pripojené k mikrokontrolérom,
- Edge vrstva: lokálne spracovanie, agregácia dát, krátkodobé ukladanie a predspracovanie (filtering, kompresia),
- Brána (gateway): protokolová prekladacia vrstva, zabezpečená konektivita do internetu, lokálna politika a prípadne správa zariadení,
- Cloud/service vrstva: dlhodobé úložisko, analýza dát, UI a integračné API.

3.2 Sieťové topológie

Výber topológie závisí od scenára použitia, fyzických obmedzení a požiadaviek na spoľahlivosť.

Hviezda (star)

Najjednoduchšia topológia, kde všetky zariadenia komunikujú priamo s centrálnou bránou. Výhody: jednoduchá správa, nízka latencia pre priame príkazy. Nevýhoda: single point of failure v prípade zlyhania brány.

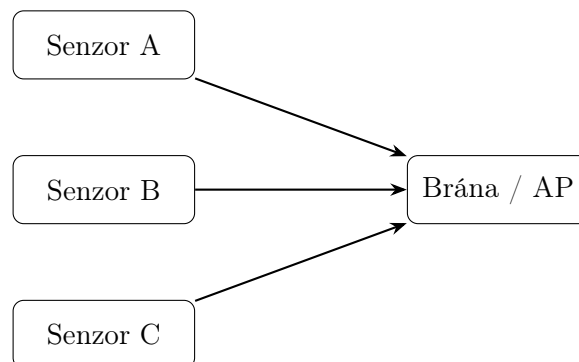


Figure 4: Hviezdicová topológia s centrálnou bránou.

Mesh

V mesh sieti zariadenia môžu smerovať správy cez viacero uzlov. Výhody: rozšíriteľnosť, odolnosť voči chybám (self-healing). Nevýhody: zložitejšie smerovanie, väčšia spotreba uzlov, komplikovanejšia správa synchronizácie.

Tree / hierarchická

Používa sa pri väčších nasadeniach, kde sú lokalizované podsiete pre efektívne smerovanie a segmentáciu. Umožňuje hierarchickú správu a zníženie broadcastov.

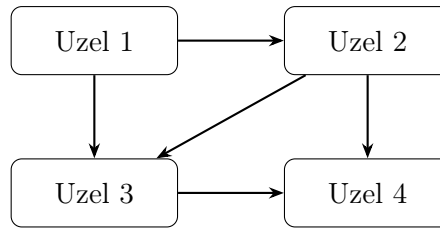


Figure 5: Ukážka mesh topológie s viacerými cestami medzi uzlami.

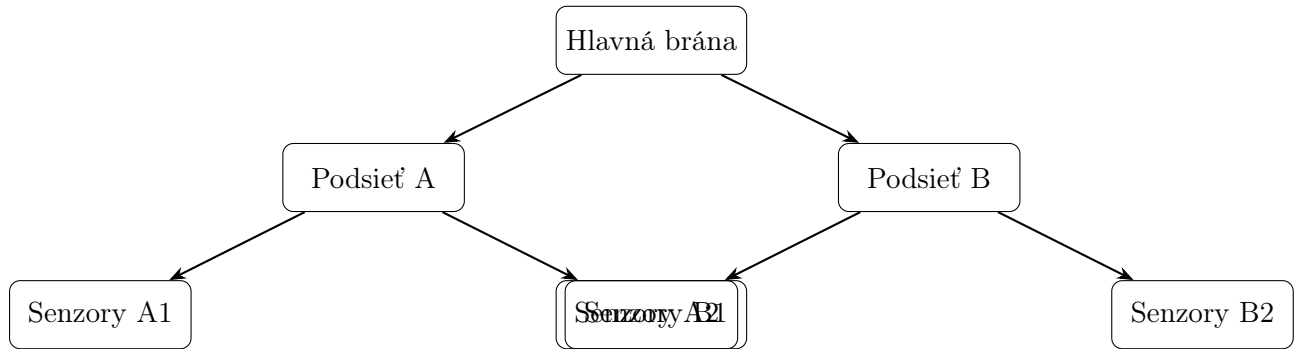


Figure 6: Hierarchická topológia s podskupinami zariadení.

Hybridné topológie

V praxi sa často používajú kombinácie, napríklad mesh v lokálnej sieti a hviezda smerom k bráne.

3.3 Praktické scenáre a odôvodnenie

Smart home

V inteligentnej domácnosti je praktická hviezda s centrálnym Wi-Fi AP alebo domácou bránou, pretože zariadenia bývajú v dosahu jednej miestnosti alebo bytu a potrebujú jednoduchú správu. Ak sú senzory na batériu a v dome je viac prekážok, často sa vyplatí Zigbee/Thread mesh, lebo umožní lepšie pokrytie a nižšiu spotrebu než Wi-Fi.

Priemyselná hala

V priemyselnej hale sa oplatí hierarchická alebo hybridná topológia, pretože zariadenia sú rozdelené do zón a medzi zónami bývajú rušenie, dlhšie trasy a rozdielne požiadavky. Lokálne podsiete môžu spracovať dáta v edge uzle a do nadradenej vrstvy posilať iba agregované výsledky, čo znižuje záťaž aj latenciu.

Rozľahlé monitorovanie

Pre poľnohospodárske parcely, environmentálne senzory alebo rozmiestnené meracie body je vhodný LoRaWAN alebo podobná nízkoenergetická hviezda s gateway, pretože zariadenia musia fungovať mesiace až roky na batériu. Tu je dôležitejší dosah a výdrž než vysoká priepustnosť, preto sa dáta posielajú len v krátkych intervaloch alebo pri zmene stavu.

Kedy zvoliť mesh a kedy nie

Mesh je výborný, keď potrebujeme premostiť prekážky a nechceme, aby jeden bod vypadol. Nehodí sa však tam, kde je kritická veľmi nízka spotreba alebo kde musíme komunikáciu udržať maximálne jednoduchú; v takom prípade je lepšia hviezda s kvalitnou bránou.

3.4 Komunikačné protokoly

Výber protokolu závisí od pasívnych a aktívnych požiadaviek:

- **Wi-Fi**: vysoká priepustnosť, jednoduché pripojenie do IP sietí, vyššia spotreba — vhodné pre napájané alebo periodicky aktívne zariadenia.
- **Bluetooth Low Energy (BLE)**: nízka spotreba, krátky dosah, vhodné pre senzory a osobné IoT.
- **LoRaWAN**: veľmi dlhý dosah pri nízkej priepustnosti, vhodné pre senzory s nízkou frekvenciou odosielania v rozľahlých oblastiach.
- **Zigbee / Thread / 6LoWPAN**: mesh-priateľské protokoly pre domácu a priemyselnú automatizáciu.
- **MQTT / MQTT-SN**: lightweight publish/subscribe protokol široko používaný v IoT; umožňuje jednoduché odosielanie telemetrie a príkazov.
- **CoAP**: REST-like protokol pre constrained zariadenia, používa UDP.

3.5 Edge vs. cloud processing

Presun časti logiky na edge (blízko zariadenia) môže znížiť latenciu a prenosovú záťaž. Pre kritické rozhodovanie alebo pri potrebe súkromnosti je vhodné vykonávať spracovanie lokálne a len agregované výsledky odosielať do cloudu.

3.6 Brána (gateway) — funkcie

Brána môže plniť viacero úloh:

- protokolová konverzia (napr. LoRaWAN → MQTT),
- zabezpečenie a autentifikácia zariadení,
- lokálne procesovanie a cacheovanie dát,
- správa aktualizácií (OTA) a centralizovaná správa zariadení.

3.7 Škálovanie a správa

Pri rozsiahlom nasadení je dôležité navrhnuť adresovanie, manažment zariadení, ukladanie telemetrie a spôsob správy konfigurácií. Manažment zahŕňa monitorovanie zdravia zariadení, alarmy a plánovanie údržby.

3.8 Bezpečnostné požiadavky

Kľúčové aspekty: zabezpečenie komunikácie (TLS/DTLS), autentifikácia zariadení, bezpečné ukladanie poverení, izolácia kritických funkcií a plán aktualizácií. Pri návrhu treba myslieť na potenciálne vektory útoku a minimalizovať rozhrania vystavené priamo internetu.

3.9 Príklady nasadení

- Smart home — Wi-Fi/BLE senzory s lokálnou bránou a cloudovým dashboardom,
- Priemyselné monitorovanie — Ethernet/CAN senzory s lokálnym edge spracovaním a redundantnými bránami,
- Dálkový monitoring — LoRaWAN senzory s nízkou frekvenciou odosielania a centralizovaným backendom.

3.10 Zhrnutie

Topológia a architektúra IoT systému musí odzrkadľovať požiadavky na dosah, spoľahlivosť, bezpečnosť a spotrebu. Výber správnych nástrojov a vrstiev výrazne uľahčí škálovanie a údržbu systémov v reálnom nasadení.

4 ESP32 – prehľad platformy

ESP32 je názov celej rodiny čipov od Espressif, ktoré sa používajú v IoT, v embedded projektoch aj v zariadeniach s bezdrôtovou komunikáciou. Spoločným menovateľom je široká podpora periférií, dostupné nástroje a silný ekosystém okolo ESP-IDF, Arduino core a MicroPythonu. Prehľad produktových rodín Espressif sa priebežne rozširuje a aktuálny katalóg zahŕňa viacero sérií pre rôzne triedy zariadení Systems 2026.

Rodiny ESP32

Najpraktickejšie je deliť platformu podľa rodín:

- **ESP32** ako pôvodná a stále rozšírená rodina s Wi-Fi a Bluetooth.
- **ESP32-S2** a **ESP32-S3** pre univerzálne Wi-Fi zariadenia, kde sa často rieši aj USB a viac periférií.
- **ESP32-C2**, **ESP32-C3**, **ESP32-C5**, **ESP32-C6** a **ESP32-C61** pre cenovo a energeticky úspornejšie RISC-V čipy.
- **ESP32-H2**, **ESP32-H21** a **ESP32-H4** pre low-power zariadenia so zameraním na BLE a IEEE 802.15.4.
- **ESP32-P4** pre výkonnejšie aplikácie s HMI, grafikou a bohatými perifériami.
- **ESP32-E22** ako konektivita-oriented co-processor v produktoch Espressif.

Hlavné vlastnosti

Aj napriek rozdielom medzi jednotlivými sériami majú zariadenia tejto rodiny spoločné typické črty:

- integrovaná bezdrôtová konektivita podľa danej série,
- bohatý set periférií pre senzoriku a riadenie,
- podporu pre spánkové režimy a nízku spotrebu,
- kompatibilný vývojový ekosystém a dostupnú dokumentáciu.

Architektúra sa líši podľa konkrétneho čipu. Staršie modely používajú Xtensa jadrá, novšie série prechádzajú na RISC-V. To je dôležité najmä pri výbere toolchainu, pri použití knižníc a pri odhadovaní výkonu.

Vývojové prostredia

K najpoužívanejším nástrojom patria ESP-IDF ako oficiálny framework, Arduino core for ESP32, PlatformIO a tiež MicroPython. Vo väčších projektoch sa často volí ESP-IDF, pretože poskytuje najpriamejší prístup k perifériám, sieťovým službám a aktualizáciám firmvéru.

Periférie v praxi

ESP32 rodina je zaujímavá tým, že okrem základných GPIO ponúka aj rozhrania, ktoré sa v embedded projektoch používajú veľmi často:

- **UART** pre debugovanie a sériovú komunikáciu,
- **I2C** pre senzory a displeje,
- **SPI** pre rýchle periférie a pamäte,
- **I2S** pre audio a streamované dáta,
- **PWM / LED PWM / MCPWM** pre LED, motory a časovo presné výstupy,
- **ADC / DAC** pre analógové merania,
- **TWAI/CAN, RMT, USB** alebo touch periférie podľa konkrétnej série.

Použitie v projektoch

V projektoch sa ESP32 používa na edge processing, riadenie senzorov, bezdrôtové brány, lokálne webové rozhrania aj komunikáciu s cloudom. Vďaka tomu sa hodí na rýchle prototypovanie aj na zariadenia, ktoré potrebujú dlhodobú a stabilnú prevádzku.

Čo si z tejto kapitoly odniesť

Pre zvyšok skript je dôležité hlavne to, že ESP32 nie je jeden čip, ale platforma s viacerými rodinami a rozdielnymi prioritami. Niektoré série sú vhodné na klasické Wi-Fi zariadenia, iné sa zameriavajú na nízku spotrebu, audio, HMI alebo na siete Thread, Zigbee a BLE. Tento výber potom priamo určuje, ktoré periférie a aký štýl programovania sa v projekte oplatí použiť.

Na detailné technické parametre a príklady kódu odkazujeme na datasheet a oficiálnu dokumentáciu Systems 2020.

5 MicroPython pre vstavané systémy

MicroPython je odľahčená implementácia jazyka Python určená pre mikrokontroléry. Jeho cieľom je ponúknuť čitateľnú syntax, rýchle prototypovanie a jednoduchú prácu s perifériami aj priamo na zariadení. V projektoch s ESP32 je veľmi vhodný na výučbu, experimentovanie aj menšie IoT prototypy.

Inštalácia cez Thonny

Pre začiatočníkov a pri rýchlom nasadzovaní odporúčame použiť grafické rozhranie Thonny, ktoré obsahuje jednoduchý nástroj na nahranie MicroPython firmvéru a prácu so sériovou konzolou.

Postup (stručne):

1. Stiahnite a nainštalujte Thonny z <https://thonny.org> pre váš operačný systém.
2. Pripojte ESP32 k počítaču cez USB a spustite Thonny.
3. V Thonny prejdite do **Tools** → **Options** → **Interpreter** a v poli **Interpreter** vyberte **MicroPython (ESP32)**.
4. Vyberte správny **Port** (napr. `/dev/ttyUSB0` alebo COM port na Windows).
5. Kliknite na tlačidlo **Install or update firmware** (v niektorých verziách cez menu za možnosťou výberu interpretera). Vyberte odporúčanú verziu firmvéru alebo zadajte cestu k stiahnutému `.bin` súboru.
6. Potvrďte nahratie; Thonny vykoná potrebné kroky (erase + write) a po skončení otvorí REPL.

Thonny tiež poskytuje jednoduchý editor a možnosť nahrávať súbory priamo na zariadenie (Save → Device). To zefektívňuje prototypovanie, pretože po uložení `main.py` zariadenie automaticky spustí upravený kód po reštarte.

Základná syntax

Syntax MicroPythonu je veľmi blízka bežnému Pythonu. Rozdiel je najmä v dostupných modulloch a v práci s hardvérom. Dôležité je odsadzovanie blokov, definovanie funkcií a používanie modulov `machine`, `time` alebo konkrétnych knižníc pre senzory.

Listing 1: Minimálny príklad v MicroPythone

```
1 from machine import Pin
2 from time import sleep
3
4 led = Pin(2, Pin.OUT)
5
6 while True:
7     led.value(1)
8     sleep(0.5)
9     led.value(0)
10    sleep(0.5)
```

V praxi sa často používajú aj vetvenia, slučky, funkcie a jednoduché stavové stroje. MicroPython umožňuje čítať GPIO, ovládať PWM, pracovať s I2C/SPI zariadeniami, komunikovať po UART a v niektorých prípadoch aj používať sieťové moduly.

Silné stránky

- rýchle prototypovanie a menšie množstvo boilerplate kódu,
- čitateľná syntax vhodná na výučbu,
- priama práca s GPIO, I2C, SPI, ADC a PWM,
- interaktívny REPL pre okamžité testovanie.

Obmedzenia

- menší výkon a väčšia pamäťová náročnosť ako pri C/C++,
- obmedzený výber knižníc oproti bežnému Pythonu,
- pri väčších projektoch treba pozor na fragmentáciu pamäte a dlhé slučky.

Nasadenie

MicroPython sa používa na ESP32, ESP8266 a na ďalších mikrokontroléroch, kde je dôležitý rýchly vývoj alebo jednoduchá údržba kódu. Pri náročnejších častiach možno kombinovať Pythonovú logiku s natívnymi modulmi alebo presunúť kritické časti do C.

Pre dokumentáciu a referencie pozri [MicroPython docs contributors 2024](#).

6 Príklady podľa zadania

V tejto kapitole sú príklady popísané sekvenčne po logických blokoch. Pri každej ukážke najprv stručne popíšem cieľ a motív riešenia, potom rozoberiem jednotlivé kroky, na ktoré treba pri implementácii dbať, a nakoniec uvádzam skrátený kódový výrez v poradí, v akom sa program číta. Texty obsahujú poznámky o možných chybách, predpokladoch o zapojení a návrhoch, ako daný blok rozšíriť.

7 Servisný test dosky

Tento príklad slúži ako rýchly self-test po flashnutí zariadenia. Cieľom je overiť základné periférie (displej, LED, senzory, tlačidlá, bzučiak) a získať rýchlu spätnú väzbu o tom, či sú piny a moduly správne zapojené. V texte uvádzam, ktoré merania sú deterministické (napr. ADC čítanie) a ktoré závisia na externých podmienkach (napr. svetelnosť), ako aj tipy na ladenie (kontrola napájania, sériový výstup, jednoduché multimeter kontroly).

Mapovanie pinov a zber výsledkov

Najprv si nadefinujem piny, ktoré program používa, aby bolo jednoduché upraviť zapojenie podľa konkrétnej dosky. Zoznam `results` slúži ako jednotný formát pre neskoršie zobrazenie výsledkov a uľahčuje rozšírenie testov alebo odoslanie reportu cez sériový port.

```
1 from machine import Pin, PWM, I2C, ADC
2 import time
3
4 PIN_I2C_SDA = 1
5 PIN_I2C_SCL = 2
6 PIN_RGB_R = 7
7 PIN_RGB_G = 5
8 PIN_RGB_B = 3
9 results = []
```

Pomocné funkcie pre výsledky

Pomocné funkcie izolujú opakujúcu sa logiku: `add_result()` formátuje a ukladá výsledok testu (názov, stav, poznámka), čo zjednoduší tlač správy alebo export. `ask_yes_no()` sa používa na manuálne overenie vizuálnych komponentov (napr. či OLED skutočne zobrazuje text) a ukážem, prečo je vhodné mať takýto blok oddelene pre jednoduché testovanie.

```
1 def add_result(name, ok, note=""):
2     state = "PASS" if ok else "FAIL"
3     results.append((name, state, note))
4
5 def ask_yes_no(question):
6     while True:
7         ans = input(question + " [y/n]: ").strip().lower()
8         if ans in ("y", "yes", "a", "ano"):
9             return True
10        if ans in ("n", "no", "nie"):
11            return False
```

Test OLED, RGB LED a WS2812

V tejto časti sa zameriame na vizuálnu spätnú väzbu: krátky text na OLED potvrdí komunikáciu po I2C, diskretná RGB LED overí základnú logiku výstupov a WS2812 reťazec demonštruje

adresovateľné LED a správne časovanie dátového signálu. Poznámka: pri WS2812 je kritické správne napájanie a dostatočné časovanie; v texte spomínam ako testovať po jednom pixeli a na čo si dávať pozor pri viacnásobnom píne.

```
1 def test_oled():
2     i2c = I2C(0, sda=Pin(PIN_I2C_SDA), scl=Pin(PIN_I2C_SCL), freq
3         =400000)
4     oled = ssd1306.SSD1306_I2C(128, 64, i2c)
5     oled.text("SELF TEST", 0, 12)
6     oled.show()
7
8 def test_rgb_discrete():
9     r = Pin(PIN_RGB_R, Pin.OUT)
10    g = Pin(PIN_RGB_G, Pin.OUT)
11    b = Pin(PIN_RGB_B, Pin.OUT)
12
13 def test_ws2812():
14     np = neopixel.NeoPixel(Pin(PIN_LED_DATA), WS2812_COUNT)
15     np[0] = (255, 0, 0)
16     np.write()
```

Test bzučiča, tlačidiel, fotorezistora a DHT

Tu prichádzajú testy, ktoré zahŕňajú časovanie (bzučič), čítanie digitálnych vstupov (tlačidlá), analógové meranie (fotorezistor cez ADC) a sekvenčné meranie (DHT senzor). V texte popisujem očakávané rozsahy hodnôt, ako si nastaviť debounce pre tlačidlá a prečo pri DHT senzore potrebujeme čakať medzi meraniami.

```
1 def test_buzzer():
2     bz = PWM(Pin(PIN_BUZZER), freq=1000, duty=0)
3     for f in (523, 659, 784):
4         bz.freq(f)
5         bz.duty(512)
6
7 def test_buttons():
8     buttons = [("USR1", Pin(PIN_BTN_1, Pin.IN)), ...]
9
10 def test_photo():
11     adc = ADC(Pin(PIN_PHOTO))
12     v = adc.read()
13
14 def test_dht():
15     sensor = dht.DHT22(Pin(PIN_DHT))
16     sensor.measure()
```

Spustenie testov a súhrn

V hlavnej časti je cyklus, ktorý postupne vykoná jednotlivé testy a zapíše výsledky do 'results'. Toto miesto je ideálne na doplnenie logovania do súboru alebo na odosielanie výsledkov cez sériový port/HTTP. V texte tiež navrhujem, ako pridať timeouty a opätovné pokusy pre nestabilné periférie.

```
1 for fn in (test_oled, test_rgb_discrete, test_ws2812,
2           test_buzzer, test_buttons, test_photo, test_dht):
3     fn()
4
5 print("=== SUMAR ===")
```

```
6 for name, state, note in results:
7     print(name, state, note)
```

8 Interakcia s tlačidlami: hra hľadania

Tento príklad demonštruje praktické použitie tlačidiel ako vstupu pre jednoduchú hru, pričom sa rieši detekcia stlačení, tvorba pseudo-náhody z kombinácie stlačení a spätná väzba užívateľovi. V texte uvádzam, ako spracovať bouncing, prečo nie je vhodné použiť ťažký výpočet v IRQ a ako testovať, či sú piny správne priradené.

Tlačidlá a základné konštanty

V tejto fáze priradíme fyzické piny k logickým menám, čo zvyšuje čitateľnosť kódu a uľahčuje zmenu zapojenia. Tiež vysvetľujem, kedy treba aktivovať vnútorné pull-up alebo pull-down rezistory a čo očakávať, keď je logická úroveň invertovaná (závisí to od zapojenia tlačidla).

```
1 from machine import Pin
2 import time
3
4 BTN1 = Pin(39, Pin.IN)
5 BTN2 = Pin(37, Pin.IN)
6 BTN3 = Pin(35, Pin.IN)
7 BTN4 = Pin(33, Pin.IN)
```

Generovanie náhodného čísla z tlačidiel

Tento jednoduchý spôsob generovania náhody čerpá entropiu zo stavu tlačidiel pri štarte (alebo z počtu stlačení). Vysvetlím, prečo to postačuje pre jednoduchú hru, ale nie je vhodné pre kryptografické účely, a ako by sa dalo rozšíriť o ďalšie zdroje náhody (ADC šum, čas medzi stlačeniami).

```
1 def get_seed():
2     seed = 0
3     if BTN1.value() == 0:
4         seed += 1
5     if BTN2.value() == 0:
6         seed += 2
7     return seed
```

Herný cyklus

V hlavnej slučke program číta vstup od používateľa, porovnáva ho s tajným číslom a dáva priebežnú spätnú väzbu. Popisujem možnosti, ako pridať časový limit, skórovací systém alebo vizuálnu odozvu cez LED, a tiež navrhujem, ako spraviť vstup robustnejším (validácia, ošetrenie chýbného vstupu).

```
1 while True:
2     guess = int(input("Tipni cislo 1-4: "))
3     if guess == secret:
4         print("Spravne")
5         break
6     print("Skus znova")
```

9 Digitálne riadenie RGB LED

Ukážka ukazuje najzákladnejší prístup k ovládaniu farieb pomocou samostatných digitálnych výstupov. Vysvetľujem, kedy je tento režim dostačujúci (indikácia stavov) a kedy treba použiť PWM pre plynulé ladenie jasů.

Digitálne výstupy RGB LED

```
1 from machine import Pin
2 import time
3
4 red = Pin(7, Pin.OUT)
5 green = Pin(5, Pin.OUT)
6 blue = Pin(3, Pin.OUT)
```

Postupné prepínanie farieb

Tento jednoduchý cyklus demonštruje, ako kombinovať kanály pre základné farby. Upozorňujem na potrebu dobrého riadenia prúdového limitu a na to, že pri viacerých kanáloch sú súčty prúdov dôležité pre napájanie dosky.

```
1 while True:
2     red.value(1)
3     green.value(0)
4     blue.value(0)
5     time.sleep(0.3)
```

10 PWM riadenie jasu RGB LED

V tejto kapitole prechádzame k jemnejšiemu ovládaniu pomocou PWM. Vysvetlím vzťah medzi frekvenciou PWM a flicker-om, typické rozsahy pre ESP32 a dôvody, prečo mať rovnomerné rozdelenie časových kvant pre jednotlivé kanály.

PWM kanály pre RGB LED

Každá farba má vlastný PWM kanál.

```
1 from machine import Pin, PWM
2
3 r = PWM(Pin(7), freq=1000)
4 g = PWM(Pin(5), freq=1000)
5 b = PWM(Pin(3), freq=1000)
```

Nastavenie hodnôt jasu

Funkcia prijíma hodnoty jasu v očakávanom intervale a mapuje ich na duty cycle PWM. Popíšem konverziu rozsahov (napr. 0–255 na 0–1023) a tipy, ako zabrániť preťaženiu GPIO pri náhlom zvýšení jasu.

```
1 def set_rgb(r_val, g_val, b_val):
2     r.duty(r_val)
3     g.duty(g_val)
4     b.duty(b_val)
```

11 Prerušená a meranie dĺžky stlačenia

Ukážka ilustruje použitie prerušenia (IRQ) na efektívnu detekciu stlačenia bez náročného polling-u v hlavnej slučke. V texte rozoberiem debounce stratégie, bezpečnosť pri práci s globálnymi premennými v IRQ kontexte a prečo by sa v obsluhu prerušenia nemali volať pomalé funkcie.

Stav a časovanie pre IRQ

```
1 from machine import Pin, I2C
2 import time
3
4 press_started = None
5 last_duration = 0
```

IRQ obsluha s debounce

IRQ handler by mal byť krátky: iba zaznamenať čas začiatku alebo konca stlačenia a nastaviť príznak. Vysvetlím, ako merať čas pomocou `time.ticks_ms()` a prečo je bezpečnejšie spracovať dĺžku tlačenia v hlavnej slučke, nie priamo v handleri.

```
1 def make_irq_handler(name):
2     def handler(pin):
3         global press_started, last_duration
4         if pin.value() == 0:
5             press_started = time.ticks_ms()
6         else:
7             last_duration = time.ticks_diff(time.ticks_ms(),
8                                             press_started)
9     return handler
```

Pripojenie handlerov na tlačidlá

```
1 btn1 = Pin(39, Pin.IN)
2 btn1.irq(trigger=Pin.IRQ_FALLING | Pin.IRQ_RISING,
3          handler=make_irq_handler("USR1"))
```

Zobrazenie posledného stlačenia

Program priebežne ukazuje poslednú nameranú dĺžku stlačenia.

```
1 while True:
2     print("Posledne stlacenie:", last_duration, "ms")
3     time.sleep(0.2)
```

12 Sériové LED WS2812B

Adresovateľný LED pásik (WS2812B) ilustruje protokol, kde sú dáta odosielané v presných časových oknách. Popíšem obmedzenia prenosu, potrebu presného nastavenia času a rady na napájanie viacerých pixelov (spoločná zem, kapacitné vyhladenie napätia).

NeoPixel objekt pre WS2812B

```
1 from machine import Pin
2 import neopixel
3
4 np = neopixel.NeoPixel(Pin(4), 12)
```

Nastavenie farieb

Vysvetlím, ako funguje buffer pixelov a prečo je dôležité volať `np.write()` až po nastavení všetkých hodnôt. Spomeniem tiež, že niektoré implementácie vyžadujú kratšie oneskorenia pred a po zápise dát.

```
1 def set_all(color):
2     for i in range(12):
3         np[i] = color
4     np.write()
```

13 PWM bzučiak a krátka melódia

Ukážka demonštruje generovanie tónov pomocou PWM, kde sa mení frekvencia namiesto duty-cyklu. Popisujem, ako nastaviť vhodnú duty pre čistý tón, prečo by sa nemal používať maximálny výkon nepretržite a ako zložiť jednoduchú melódiu bez externých knižníc.

PWM výstup pre bzučiak

```
1 from machine import Pin, PWM
2 import time
3
4 buzzer = PWM(Pin(11), freq=1000, duty=0)
```

Funkcia na prehratie tónu

Funkcia nastaví frekvenciu a krátko aktivuje výstup. V poznámke uvádzam, že pri dlhých tónoch je vhodné znižovať duty alebo vložiť malé prestávky, aby sa zabránilo prehrievaniu aktívneho bzučiaka.

Krátka melódia

```
1 for note in (523, 659, 784):
2     beep(note, 180)
3     time.sleep_ms(60)
```

14 Analógové meranie svetla

V tejto kapitole popisujem meranie odporu fotorezistoru cez ADC a postup kalibrácie, ktorý zabezpečí zmysluplné percentuálne hodnoty v rôznych svetelných podmienkach. Upozorním na offsety ADC, potrebu filtrovania a ako zmerať minimálne a maximálne hodnoty prakticky.

Inicializácia ADC

```
1 from machine import Pin, ADC
2
3
4 adc.atten(ADC.ATTN_11DB)
```

Kalibrácia minima a maxima

Kalibrácia spočíva v pokrývaní extrémov (tma vs. silné svetlo) a zapísaní týchto hodnôt, ktoré sa potom použijú na normalizáciu vstupu. V texte dávam tipy, ako vykonať viacnásobné merania a použiť klzavý priemer pre stabilnejší výsledok.

```
1 min_v = 65535
2 max_v = 0
3 v = adc.read()
```

Prepočet na percentá

Normalizácia spraví z ADC čísla ľahšie čitateľný výstup.

```
1 def normalize(value, min_v, max_v):
2     span = max_v - min_v
3     return int((value - min_v) * 100 / span)
```

15 Senzor DHT: teplota a vlhkosť

Krátka, ucelená ukážka načítania teploty a vlhkosti zo senzora DHT. Okrem základného princípu merania popisujem časové obmedzenia medzi meraniami a prípadné chyby pri čítaní, ktoré treba ošetriť (timeouty, chybové kódy).

```
1 from machine import Pin
2 import dht
3 import time
4
5 sensor = dht.DHT22(Pin(18))
6
7 while True:
8     sensor.measure()
9     print(sensor.temperature(), sensor.humidity())
10    time.sleep(2)
```

16 Wi-Fi prístupový bod a webové ovládanie

Tento príklad prepája lokálne periférie s jednoduchým webovým rozhraním cez vlastný access point. Popisujem architektúru: inicializácia periférií, tvorba HTML rozhrania pre ovládanie, bezpečnostné a spoľahlivé spracovanie požiadaviek, a návrhy na rozšírenie (autentifikácia, stavové stránky, AJAX pre aktualizácie bez reloadu).

1. Pripravím periférie a fallbacky

V tejto časti sa oboznámime so spôsobom, ako robustne inicializovať periférie: najprv skúsiť importovať voliteľné moduly a v prípade chýb poskytnúť fallback (napr. simulované čítania). Uvádzam príklady, čo robiť, keď senzor nie je prítomný, aby server neskončil s chybou.

```
1 import socket, time, network
2 from machine import Pin, PWM, I2C
3
4 try:
5     import dht
6 except ImportError:
7     dht = None
```

2. Vytvorím pomocné funkcie pre výstupy

Oddelené funkcie pre ovládanie výstupov zlepšujú čitateľnosť a uľahčujú viazanie rozhrania (mapovanie tlačidiel na akcie). Vysvetlím, ako mať čisté API pre zmenu stavu (napr. `set_rgb(name)`) a prečo je dobré mať centrálny stav zariadenia, ktorý HTML stránka vie prečítať.

```
1 def set_rgb(name):
2     colors = {"red": (1, 0, 0), "green": (0, 1, 0), "off": (0, 0, 0)}
3     }
4     r, g, b = colors.get(name, (0, 0, 0))
5     red.value(r)
6     green.value(g)
7     blue.value(b)
```

3. Vygenerujem HTML stránku

HTML stránka je minimálna a slúži na demonštráciu ovládania: tlačidlá odosielaajú query parametre a stránka zobrazuje aktuálne merania. Uvádzam, ako zaradiť CSRF bezpečnosť minimalizovaným spôsobom (jednorazové tokeny) a ako vylepšiť používateľské rozhranie jednoduchým CSS/JS.

```
1 def html_page(temp, hum):
2     return """<!doctype html>
3     <html>
4     <body>
5     <a href='/?rgb=red'>Red</a>
6     <a href='/?buzz=1'>Pipni</a>
7     </body></html>"""
```

4. Spustím access point

Spustenie AP modeluje režim, keď zariadenie poskytuje vlastnú sieť bez potreby existujúcej infraštruktúry. Popisujem, kedy je vhodné AP vs. STA režim, ako nastaviť jednoduché heslo a čo očakávať pri obmedzenom dosahu/kanálových konfliktoch.

```
1 def start_ap():
2     ap = network.WLAN(network.AP_IF)
3     ap.active(True)
4     ap.config(essid=AP_SSID, password=AP_PASSWORD, authmode=3)
```

5. Obslúžim HTTP požiadavky

Server prijíma jednoduché GET požiadavky a na základe parametrov vykonáva akcie (zapnúť LED, prehriať bzučiak). Upozorňujem na potrebu parsovania a validácie parametrov, správnu uzávierku soketov a na to, že server by mal byť tolerantný k neúplným požiadavkám.

```
1 while True:
2     conn, _ = s.accept()
3     req = conn.recv(1024)
4     if "rgb" in req:
5         set_rgb(req["rgb"])
6     if req.get("buzz") == "1":
7         buzz()
```

17 Odosielanie dát do ThingSpeak

Praktický príklad integrácie so vzdialenou cloudovou službou. Popisujem, ako efektívne bufferovať merania, zaistiť opakované odosielanie pri výpadkoch siete a minimalizovať spotrebu energie pri pravidelnom odosielaní dát.

1. Pripravím Wi-Fi a senzor

```
1 import network, time, urequests
2 from machine import Pin
3 import dht
4
5 sensor = dht.DHT22(Pin(18))
```

2. Pripojím sa do siete

Keď sa Wi-Fi pripojenie nepodarí, program to vie zistiť a skúsiť znova.

```
1 def wifi_connect():
2     wlan = network.WLAN(network.STA_IF)
3     wlan.active(True)
4     wlan.connect(WIFI_SSID, WIFI_PASSWORD)
```

3. Načítam hodnoty a pošlem ich

ThingSpeak očakáva jednoduchý HTTP request s parametrami kanála.

```
1 def thingspeak_update(temp, hum):
2     url = THINGSPEAK_URL + "?field1=%s&field2=%s" % (temp, hum)
3     urequests.get(url).close()
```

4. Opakujem cyklus v hlavnej slučke

```
1 while True:
2     sensor.measure()
3     thingspeak_update(sensor.temperature(), sensor.humidity())
4     time.sleep(20)
```

18 MQTT publish/subscribe komunikácia

MQTT je vhodný tam, kde zariadenie musí nielen odosielať dáta, ale aj prijímať príkazy. Tento príklad ukazuje oba smery komunikácie.

1. Pripravím Wi-Fi a MQTT konfiguráciu

```
1 import network, time
2 from umqtt.simple import MQTTClient
3
4 CLIENT_ID = b"iotstich-01"
5 MQTT_BROKER = b"test.mosquitto.org"
6 TOPIC_PUB = b"iotstich/telemetry"
7 TOPIC_SUB = b"iotstich/cmd"
```

2. Pripojím sa do Wi-Fi

```
1 def wifi_connect():
2     wlan = network.WLAN(network.STA_IF)
3     wlan.active(True)
4     wlan.connect(WIFI_SSID, WIFI_PASSWORD)
```

3. Definujem MQTT callback

Callback reaguje na prichádzajúce správy a vie podľa nich zmeniť stav dosky.

```
1 def on_message(topic, msg):
2     print(topic, msg)
3     if msg == b"ping":
4         print("pong")
```

4. Publikujem telemetriu v hlavnej slučke

```
1 client = MQTTClient(CLIENT_ID, MQTT_BROKER)
2 client.set_callback(on_message)
3 client.connect()
4 client.subscribe(TOPIC_SUB)
5
6 while True:
7     client.publish(TOPIC_PUB, b"25.4,60.1")
8     client.check_msg()
9     time.sleep(10)
```

References

- contributors, MicroPython (2024). *MicroPython Documentation*. <https://docs.micropython.org>.
- Systems, Espressif (2020). *ESP32 Series Datasheet*. <https://www.espressif.com>.
- (2026). *ESP32 SoCs*. <https://www.espressif.com/en/products/socs>.
- Wolf, Frank V. (2012). *Embedded Systems: Architecture, Programming and Design*. CRC Press.